

SIMATS ENGINEERING ASSESSMENT TOOL 2

NAME: MURALI KRISHNAN N

REG NO: 192524090

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA0613

COURSE OUTCOME COVERED

CO 4: Solve optimization problems using dynamic programming and greedy strategies. (BL4, BL5)

Annexure A

Industry Problem Solving Task

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution

		industry standards			
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional,	Good documentation	Basic documentation	Poor documentation
		well-documented, and clearly presented work	and presentation	with some gaps	on and presentation

Q25. Data Center Load Balancing Optimization (Divide & Conquer)

Problem Statement: A data center must distribute incoming requests across multiple servers to maintain optimal performance. This answer analyzes how divide and conquer strategies can be applied to balance load efficiently, identifies the constraints involved (server capacity and request variability), designs a scalable load balancing algorithm, discusses real-time implementation challenges, evaluates performance in terms of latency and throughput, and interprets the results in a high-performance computing (HPC) context.

1. Understanding of the Industry Problem

Modern data centers host thousands of servers that must jointly service a continuous, unpredictable stream of client requests — web page loads, API calls, database queries, or video-streaming sessions. If requests are routed poorly, some servers become overloaded (causing high latency, timeouts, or crashes) while others sit idle, wasting capacity that was paid for. The business impact is direct: Service Level Agreements (SLAs) are violated, user experience degrades, and infrastructure cost-efficiency drops. The core engineering challenge is therefore to design a load-balancing mechanism that (a) decides, in real time, which server should handle each incoming request, and (b) can scale to a very large and dynamically changing pool of servers without the decision-making process itself becoming a bottleneck.

A naive centralized approach — where a single load balancer inspects every server's current load and picks the best one by scanning the entire list — works for small clusters but degrades linearly (or worse) as the number of servers N grows into the thousands. This is precisely where a divide-and-conquer (D&C) formulation becomes valuable: instead of solving "find the best of N servers" as one large problem, D&C recursively partitions the server pool into halves, solves the placement/selection sub-problem on each half independently, and combines the partial results, turning a linear scan into a logarithmic decision process.

2. Constraints Identified

- Server capacity constraint: each server has a finite CPU, memory, and network-bandwidth ceiling beyond which response times rise sharply (queueing effects), so the algorithm must track a live "load metric" per server and never route beyond a safe threshold.
- Request variability constraint: incoming requests are heterogeneous — some are lightweight (a static file), others are heavyweight (a video-transcoding job) — so a purely round-robin assignment (ignoring request cost) is insufficient.
- Real-time constraint: the routing decision for one request must be made in microseconds to milliseconds, so the divide-and-conquer decision structure must be maintainable incrementally rather than rebuilt from scratch for every request.
- Fault-tolerance constraint: servers can fail or be added/removed (auto-scaling) at any time, so the partition structure must support dynamic insertion and deletion without a full rebalance.
- Fairness / SLA constraint: high-priority tenants or requests may need guaranteed minimum resources, adding a weighting dimension to the partitioning criterion.

3. Application of Engineering Concepts — Why Divide & Conquer

Divide and conquer is the natural fit here because the load-balancing decision problem exhibits the same structural property that makes D&C effective elsewhere: the problem of finding the least-loaded (or best-fit) server among N servers can be split into two independent sub-problems of size $N/2$ each, and the answer to the whole problem is simply obtained by comparing the two sub-answers — a constant-time combine step. This mirrors the classic D&C recurrence and is conceptually close to a tournament / segment-tree style selection, which is why HPC load balancers frequently implement this idea using a segment tree or a balanced binary structure over the server pool.

Divide: Partition the set of servers $S = \{s_1, s_2, \dots, s_N\}$ into two (or more) contiguous or hashed sub-groups, S_{left} and S_{right} , each maintained by its own sub-balancer or its own subtree in a segment tree.

Conquer: Recursively determine the best candidate server in S_{left} and the best candidate in S_{right} , using each server's current load metric $L(s_i) = (\text{active connections, or weighted resource usage})$.

Combine: Compare the best candidate from the left half against the best candidate from the right half and propagate the overall best candidate upward; the request is finally routed to this server and its load metric is updated, which is then propagated back down the same recursive path (an update operation identical to a segment-tree point update).

This design also elegantly incorporates the request-variability constraint: instead of a plain "least connections" value, $L(s_i)$ can be a weighted score that accounts for the incoming request's estimated cost, so the combine step is comparing predicted post-assignment load rather than raw current load.

4. Solution Design

The proposed solution models the server pool as a segment tree (a classical divide-and-conquer data structure) of size N , where each leaf node stores the live load of one server, and each internal node stores the minimum load among its two children along with a pointer to which leaf achieves that minimum. This allows both querying the globally least-loaded server and updating one server's load after assigning it a request to be performed in $O(\log N)$ time instead of $O(N)$.

State representation: Leaf $i \rightarrow (\text{server_id} = i, \text{load} = L(i))$. Internal node $v \rightarrow \min(\text{load of left child}, \text{load of right child})$, and $\text{best_server}(v) = \text{whichever child achieves that minimum}$.

Recurrence relation: $\text{best}(v) = \text{if } v \text{ is a leaf: } (v, L(v)); \text{ else: the pair with smaller load among } \text{best}(\text{left}(v)) \text{ and } \text{best}(\text{right}(v))$.

Pseudocode for the two operations required at every incoming request:

```
FUNCTION query_best(node): IF node is leaf: RETURN (node.server_id, node.load) left_best = query_best(node.left) right_best = query_best(node.right) RETURN the one with the smaller load
FUNCTION assign_request(request): (server_id, load) = query_best(root) route request to server_id new_load = load + estimated_cost(request) update_leaf(server_id, new_load) // propagate change up the tree, O(log N) RETURN server_id
```

Handling dynamic scaling (constraint 4) is a direct benefit of the segment-tree formulation: adding or removing a server corresponds to inserting or deleting a leaf and rebuilding only the $O(\log N)$ ancestor chain of that leaf, not the entire tree — a property a flat array-scan approach cannot offer without an $O(N)$ rebuild.

For very large data centers, the same divide-and-conquer idea is applied hierarchically at two levels: (i) a top-level D&C balancer distributes traffic across geographically distinct data-center clusters, and (ii) a cluster-level D&C balancer (the segment tree above) distributes traffic within a cluster across individual servers, which keeps each individual decision tree shallow and bounded even as total server count grows into the tens of thousands.

5. Suggested Tools and Technologies for Implementation

- Load balancer software: NGINX Plus, HAProxy, and Envoy support custom weighted/least-connection routing policies that can be extended to implement the segment-tree logic above.
- Orchestration: Kubernetes Services with custom schedulers, combined with the Horizontal Pod Autoscaler, to reflect the dynamic add/remove-server constraint automatically.

- Cloud-native load balancing: AWS Elastic Load Balancing / Application Load Balancer, Google Cloud Load Balancing, and Azure Load Balancer, which already expose least-outstanding-request and weighted routing algorithms that approximate this design at scale.
- Monitoring & telemetry: Prometheus and Grafana to continuously feed the live load metric $L(s_i)$ that the segment tree depends on.
- Prototyping / simulation: Python (with a simple segment-tree/heap implementation) or a discrete-event simulator such as CloudSim to validate the algorithm's behaviour before production deployment.

6. Time and Space Complexity Analysis

Let N denote the number of servers in the pool.

- Naive linear scan (baseline): $O(N)$ time per request to find the least-loaded server, $O(N)$ space to store loads.
- Segment-tree divide-and-conquer design: $O(\log N)$ time per request for both the query (find best server) and the update (reflect new load), and $O(N)$ space for the tree itself ($2N - 1$ nodes for N leaves).
- Adding/removing a server: $O(\log N)$ time, versus $O(N)$ for a full rebuild in the naive approach.
- Hierarchical two-level extension: if a data center has C clusters of N/C servers each, routing cost becomes $O(\log C + \log(N/C)) = O(\log N)$, i.e., the asymptotic advantage is preserved even when the design is split across geography.

This logarithmic behaviour is the central engineering payoff: at $N = 10,000$ servers, $\log_2(N)$ is approximately 14, meaning the D&C balancer performs on the order of 14 comparisons per request versus up to 10,000 comparisons for the naive scan — a reduction of roughly three orders of magnitude in decision-making work per request.

7. Python Implementation of the Divide-and-Conquer Load Balancer

The following Python program implements the segment-tree based load balancer described above. It supports the two core operations derived from the divide-and-conquer recurrence — `query_best()` (find the least-loaded server) and `update()` (reflect a server's new load after assignment) — both in $O(\log N)$ time, along with dynamic add/remove of servers and a small driver simulation that assigns a stream of requests.

```
class LoadBalancerSegmentTree:
    """
    Divide-and-conquer (segment tree) based data-center load balancer.
    Each leaf holds one server's current load; each internal node
    stores the (server_id, load) pair of whichever child has the
    smaller load -- this is the 'combine' step of D&C.
    """

    def __init__(self, num_servers):
        self.n = num_servers
        self.size = 1
        while self.size < self.n:
            self.size *= 2
        # tree[i] = (server_id, load); -1 marks an unused padding leaf
        self.tree = [(-1, float("inf"))] * (2 * self.size)
        for i in range(self.n):
            self.tree[self.size + i] = (i, 0)
        for i in range(self.size - 1, 0, -1):
            self._merge_up(i)

    def _merge_up(self, i):
        left, right = self.tree[2 * i], self.tree[2 * i + 1]
        self.tree[i] = left if left[1] <= right[1] else right

    def query_best(self):
        """DIVIDE & CONQUER 'conquer' step:  $O(\log N)$  best-server lookup."""
        return self.tree[1] # root already holds the global best

    def update(self, server_id, new_load):
        """Point update after assignment;  $O(\log N)$ ."""
        i = self.size + server_id
        self.tree[i] = (server_id, new_load)
        i //= 2
        while i >= 1:
            self._merge_up(i)
            i //= 2

    def get_load(self, server_id):
        return self.tree[self.size + server_id][1]

    def assign_request(self, estimated_cost):
        """Route one incoming request to the currently best server."""
        server_id, current_load = self.query_best()
        new_load = current_load + estimated_cost
        self.update(server_id, new_load)
```

```

        return server_id, new_load

def add_server(self):
    """Dynamic scale-out: register a new empty server, O(log N)."""
    new_id = self.n
    self.n += 1
    if self.n > self.size:
        self._rebuild(self.size * 2)
    self.update(new_id, 0)
    return new_id

def remove_server(self, server_id):
    """Dynamic scale-in: take a server out of rotation, O(log N)."""
    self.update(server_id, float("inf"))

def _rebuild(self, new_size):
    old_loads = [self.get_load(i) for i in range(self.n - 1)]
    self.size = new_size
    self.tree = [(-1, float("inf"))] * (2 * self.size)
    for i, load in enumerate(old_loads):
        self.tree[self.size + i] = (i, load)
    for i in range(self.size - 1, 0, -1):
        self._merge_up(i)

if __name__ == "__main__":
    NUM_SERVERS = 8
    balancer = LoadBalancerSegmentTree(NUM_SERVERS)

    # Simulated stream of incoming requests with varying cost
    incoming_requests = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]

    for req_id, cost in enumerate(incoming_requests):
        server_id, new_load = balancer.assign_request(cost)
        print(f"Request {req_id:2d} (cost={cost}) -> Server {server_id} "
              f"(new load={new_load})")

    print("\nFinal load distribution:")
    for s in range(NUM_SERVERS):
        print(f"  Server {s}: load = {balancer.get_load(s)}")

    # Demonstrate elastic scaling (constraint: dynamic add/remove)
    new_server = balancer.add_server()
    print(f"\nScaled out -> added Server {new_server}")
    balancer.remove_server(2)
    print("Scaled in -> Server 2 removed from rotation")

```

Complexity check: `query_best()` is $O(1)$ (root lookup) and `update()` walks one root-to-leaf path, so it is $O(\log N)$; `add_server()` and `remove_server()` also cost $O(\log N)$ except for the rare doubling rebuild, which is amortised $O(1)$ per insertion overall. This matches the theoretical analysis in Section 6 and can be benchmarked directly by timing `assign_request()` across increasing values of `NUM_SERVERS`.

8. Performance Evaluation: Latency and Throughput

Two metrics matter most in this industry context: (a) decision latency — the time the balancer itself takes to pick a server, which directly adds to end-to-end response time, and (b) system throughput — the number of requests per second the whole data center can sustain before queues start to grow unboundedly.

Servers (N)	Naive scan decision cost	D&C (segment tree) decision cost
100	~100 comparisons	~7 comparisons
1,000	~1,000 comparisons	~10 comparisons
10,000	~10,000 comparisons	~14 comparisons
100,000	~100,000 comparisons	~17 comparisons

Because the decision cost grows only logarithmically, the balancer's own overhead stays a negligible, near-constant fraction of the total request-handling time even as the data center scales out, which in turn means throughput is limited by actual server processing capacity rather than by the load-balancing layer itself — precisely the scalability property required in production HPC and cloud environments. In practice this also reduces tail latency (p99 response time), because the D&C balancer reacts to load imbalance within a single routing decision instead of periodically re-scanning the whole pool.

9. Real-Time Implementation Challenges

- Consistency vs. speed trade-off: under very high request rates, many requests may read the segment tree concurrently while others are updating it; a fully locked tree becomes a new bottleneck, so practical systems use lock-free or sharded segment trees (one shard per rack) with periodic global rebalancing.
- Staleness of load metric: network delay between a server reporting its load and the balancer using that value means decisions are always made on slightly outdated information, which can cause short-lived herd effects (many requests briefly routed to the same "apparently free" server).
- Heterogeneous hardware: servers with different CPU/memory specifications need the load metric to be normalized (e.g., percentage utilization rather than raw connection count) before being compared in the tree.
- Cold-start and warm-up: a newly added server should not be flooded immediately; a gradual ramp-up weight is often added on top of the base D&C logic.

10. Interpretation of Results in Industry Context

For a high-performance computing or large-scale cloud data center, the divide-and-conquer load-balancing design converts an operation that would otherwise scale poorly (linear in the number of servers) into one that scales logarithmically, which is the difference between a load balancer that remains responsive at cluster sizes of a few dozen machines and one that remains responsive at cluster sizes of hundreds of thousands of machines. This directly supports three business-level outcomes: (1) SLA compliance, because per-request routing overhead stays bounded and predictable regardless of growth; (2) cost efficiency, because servers are kept closer to their target utilization band instead of some being idle and others overloaded; and (3) elastic scalability, because the $O(\log N)$ add/remove-server operation matches the dynamic, auto-scaling nature of modern cloud infrastructure. The design therefore satisfies both the algorithmic requirement (efficient optimal-substructure based selection, the hallmark of divide-and-conquer) and the industry requirement (real-time responsiveness at data-center scale).

11. Conclusion

The data center load-balancing problem is well suited to a divide-and-conquer solution because the task of selecting the best server among many decomposes cleanly into independent sub-selections that can be combined in constant time, exactly the structural signature D&C is designed to exploit. Modelling the server pool as a segment tree yields $O(\log N)$ request routing and $O(\log N)$ dynamic scaling, a substantial improvement over the $O(N)$ cost of naive scanning, while remaining compatible with industry-standard tools such as NGINX, HAProxy, Envoy, and Kubernetes-based orchestration. The approach directly addresses the stated constraints of server capacity, request variability, and real-time responsiveness, making it a practically deployable strategy for large-scale, high-performance computing environments.

